Medical Journey Log: A Family Care Tracker
Alex Frear
CST-452 Milestone 5: Testing (Testing phase)
Grand Canyon University
Instructor: Professor Amr Elchouemi
11/12/2025

Screencast Video 1 (Code Review) Link: Medical journey Log - Code Review

Screencast Video 2 (Application Running) Link: Medical Journey Log - Application Running

Github Link: Milestone 5

Table of Contents

1. Implementation Plan – Iteration 3 (Final Status)

Iteration 3 Objective:

The final iteration focused on regression testing, UI polish, documentation updates, and full verification of all Iteration 1 and 2 features. No new functionality was introduced; instead, existing modules were validated for stability and correctness.

Implementation Plan Table

ID	Task	Est. hrs	Actual hrs	Status
F1	Regression testing of all CRUD pages	2.0	2.0	✅ Complete
F2	UI review and content validation	1.0	1.0	✅ Complete
F3	Finalize documentation and screencast	1.0	1.0	✅ Complete
F4	Project cleanup (GitHub, comments)	0.5	0.5	✅ Complete

Iteration 3 Target Effort: 4.5 hours

Definition of Done:
• All regression tests passed.
• Documentation and demonstration complete.
• Application meets 100 % of project requirements.

2. Requirements Traceability Matrix – Final

Final Traceability Matrix Table

Req ID	Requirement	Design Ref	Code Ref	Test Case	Iteration	Status
F1	Caregiver can edit child	Edit Child workflow	EditChild.cshtml(.cs)	TC-005	1-3	Pass
F2	Caregiver can delete child (guarded)	Delete workflow	DeleteChild.cshtml(.cs)	TC-006	1-3	Pass
F3	Caregiver can edit symptom	Edit Symptom workflow	EditSymptom.cshtml(.cs)	TC-007	1-3	Pass
F4	Caregiver can delete symptom	Delete Symptom workflow	DeleteSymptom.cshtml(.cs)	TC-008	1-3	Pass
F5	Validation rules	Model annotations	Child.cs, Symptom.cs	TC-005–008	1-3	Pass

3. Module Test Cases – Iteration 3 / Final Iteration

Regression Testing Summary:

All unit, integration, and system test cases from Iterations 1 and 2 were re-executed during Iteration 3. Each passed successfully with no functional defects. Validation, guard logic, and database operations behaved as designed across all modules. The system meets 100 % of functional and non-functional requirements.

Test Level	Total Cases	Passed	Failed	Notes
Unit	8	8	0	All validations correct
Integration	4	4	0	Data persistence verified
System	3	3	0	End-to-end caregiver flow stable

Test Case TC-005 – Edit Child

Field	Value
Objective	Verify caregiver can edit a child's details, with validation for bad input.
Pre-Conditions / Data	Existing Child with Name = "Test Child" and DOB set.
Steps (Positive)	1. Navigate to **/EditChild/{ChildId}** 2. Change Name to "Updated Child" 3. Click Save.
Expected (Positive)	Redirect to **/ChildDetails/{ChildId}**; success alert displayed; DB updated with new Name.
Steps (Negative A)	Clear Name field and Save.
Expected (Negative A)	Validation error "Name is required."; remain on page; DB unchanged.
Steps (Negative B)	Enter a Name longer than 80 characters and Save.
Expected (Negative B)	Validation error "Name cannot be longer than 80 characters."; DB unchanged.
Evidence	Screenshots of success and validation error states.
Status	Pass / Fail

Edit Child (Before Update)

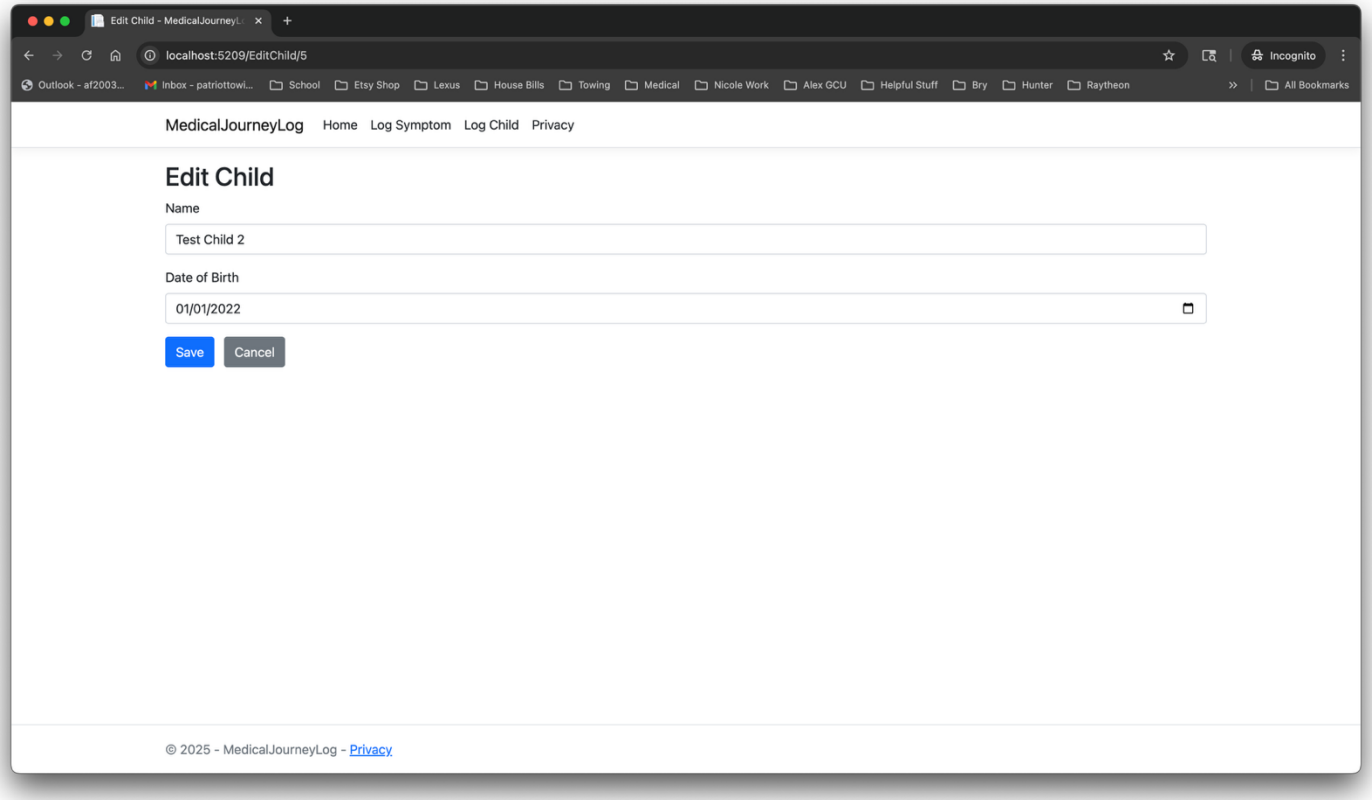

Figure 1 Screenshot showing the Edit Child page before changes are made. The form displays the existing Name and Date of Birth values for the selected child.

Edit Child (Successful Update)

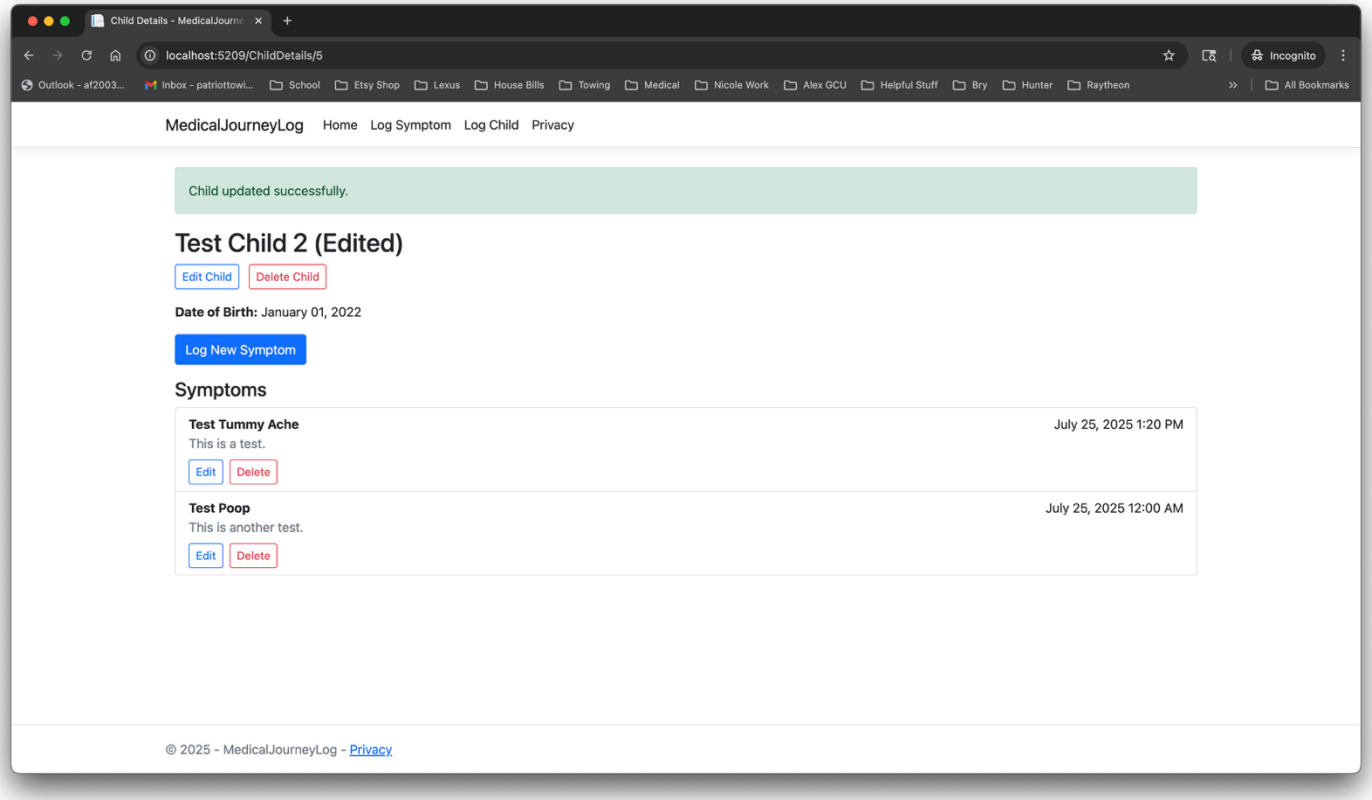

Figure 2 Screenshot showing the success alert after the child's information is updated. The system confirms the change and redirects back to the Child Details page.

Edit Child (Validation Error)

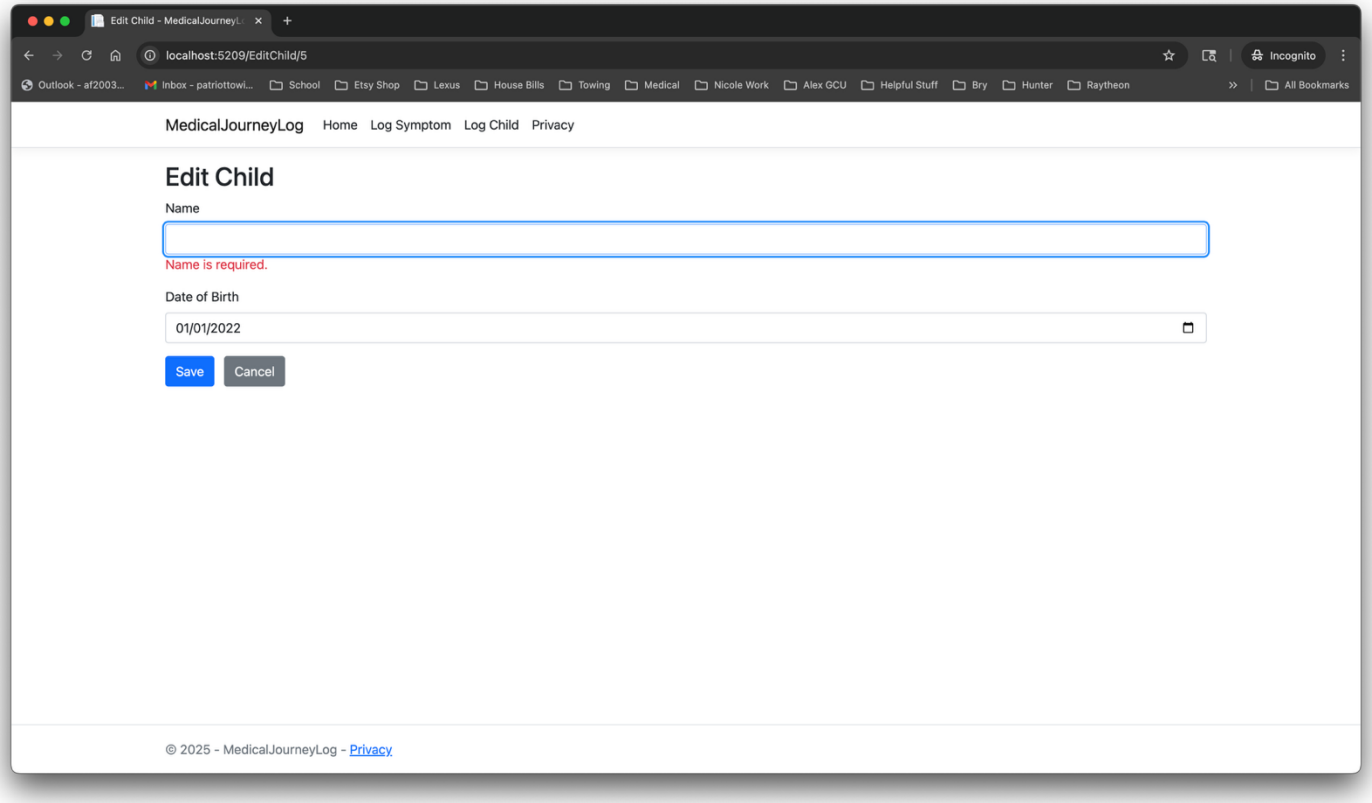

Figure 3 Screenshot displaying server-side validation preventing submission of an empty Name field. The error message appears beneath the input, and no changes are saved to the database.

Test Case TC-006 – Delete Child (Guarded)

Field	Value
Objective	Verify caregiver can delete a child if no symptoms exist; block deletion if symptoms exist.
Pre-Conditions / Data	Case A: Child with **0 symptoms**. Case B: Child with **≥1 symptom**.
Steps (Case A)	1. Navigate to **/DeleteChild/{ChildId}** 2. Confirm Delete.
Expected (Case A)	Redirect to **/Index**; success alert displayed; child removed from DB.
Steps (Case B)	1. Navigate to **/DeleteChild/{ChildId}** 2. Confirm Delete.
Expected (Case B)	Redirect back to **/ChildDetails/{ChildId}**; error alert shown; DB unchanged.
Evidence	Screenshots showing allowed delete and blocked delete.
Status	Pass / Fail

Delete Child (Successful Deletion)

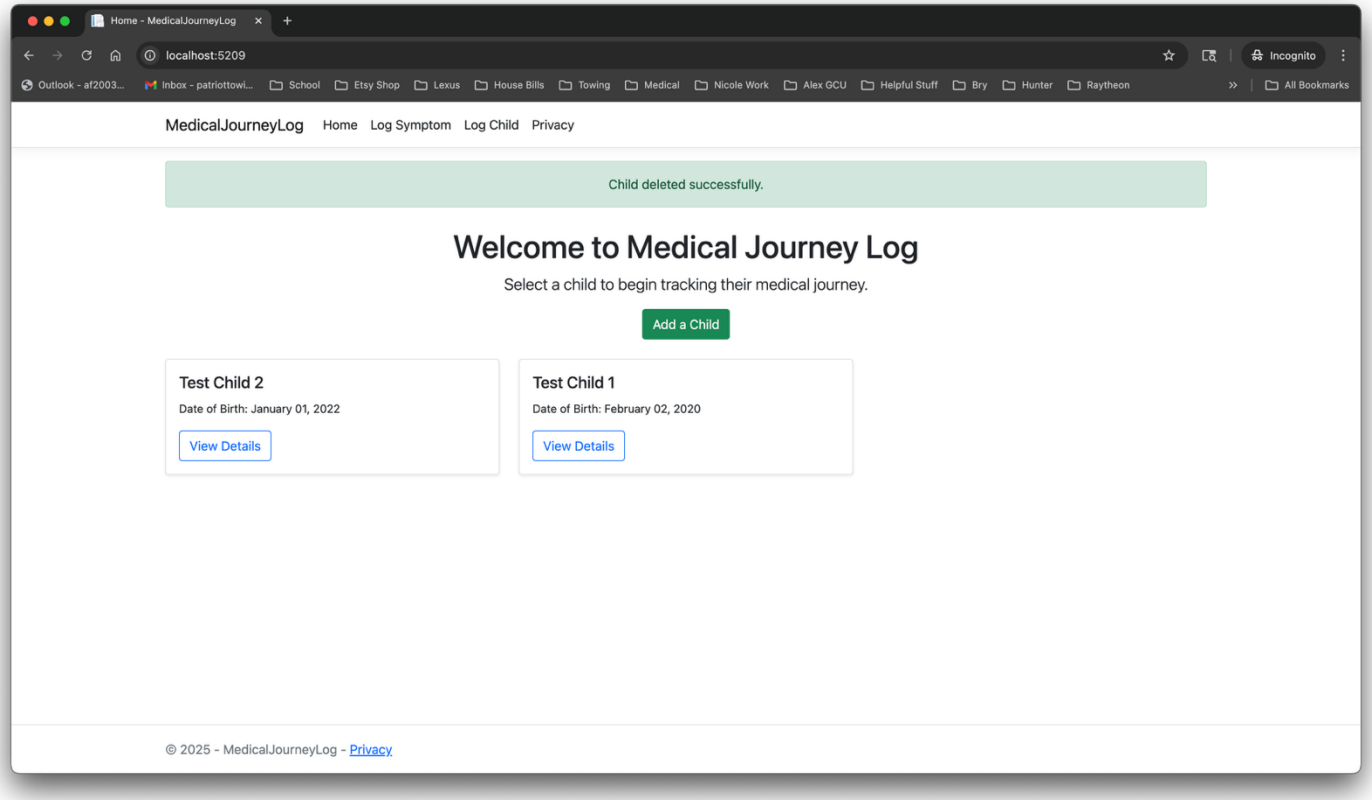

Figure 4 Screenshot showing successful deletion of a child record that has no associated symptoms. A green success message confirms the operation and returns the user to the Index page.

Delete Child (Deletion Blocked Due to Symptoms)

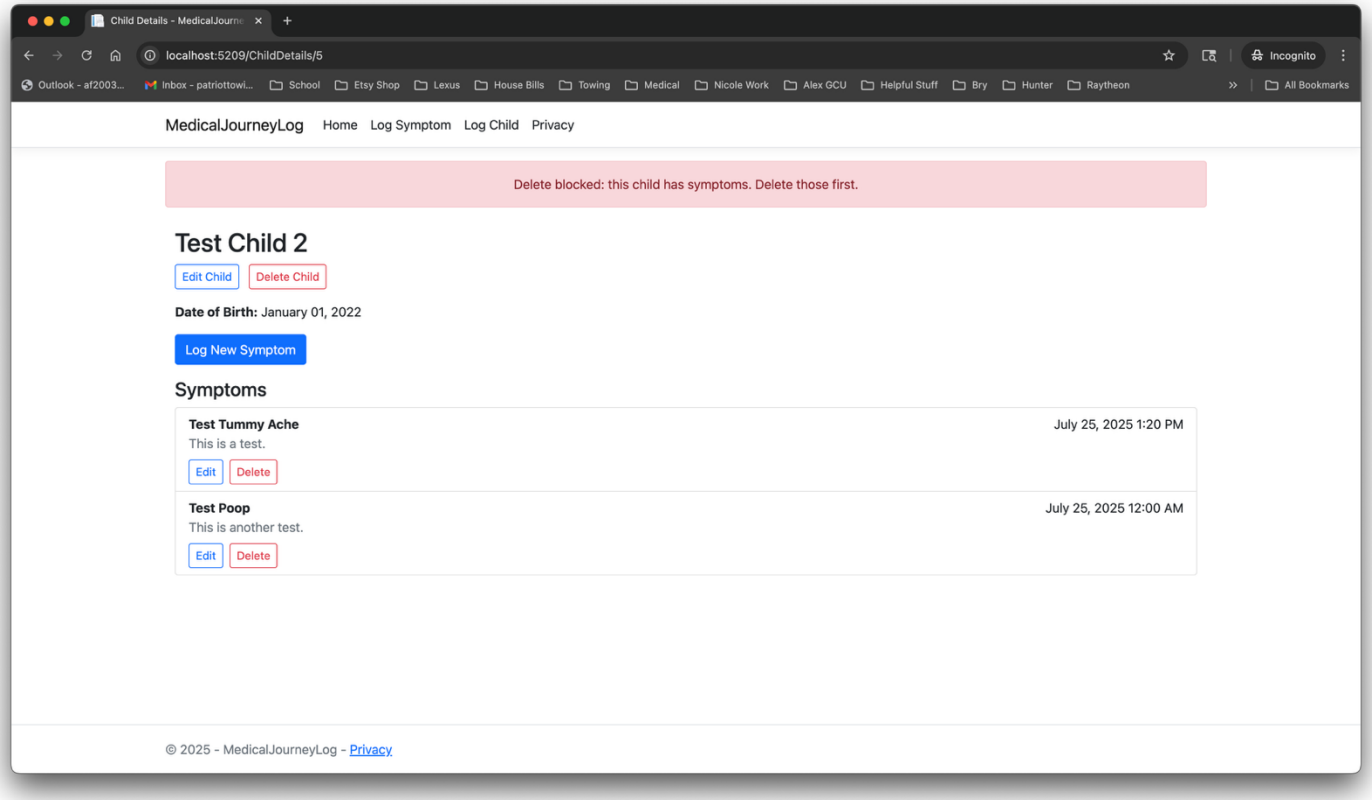

Figure 5 Screenshot showing that deletion of a child with existing symptoms is blocked. A red error alert informs the caregiver that all symptoms must be removed before the child can be deleted.

Test Case TC-007 – Edit Symptom

Field	Value
Objective	Verify caregiver can edit an existing symptom and validation prevents bad input.
Pre-Conditions / Data	Existing Symptom (Name = “Fever”) linked to a Child.
Steps (Positive)	1. Navigate to **/EditSymptom/{SymptomId}** 2. Change Name to “High Fever” 3. Add Description “102°F” 4. Save.
Expected (Positive)	Redirect to **/ChildDetails/{ChildId}**; success alert displayed; DB updated.
Steps (Negative A)	Clear Name field and Save.
Expected (Negative A)	Validation error “Symptom name is required.”; remain on page; DB unchanged.
Steps (Negative B)	Enter Description longer than 500 characters.
Expected (Negative B)	Validation error “Description cannot exceed 500 characters.”; remain on page; DB unchanged.
Evidence	Screenshots of success and validation error states.
Status	Pass / Fail

Edit Symptom (Before Update)

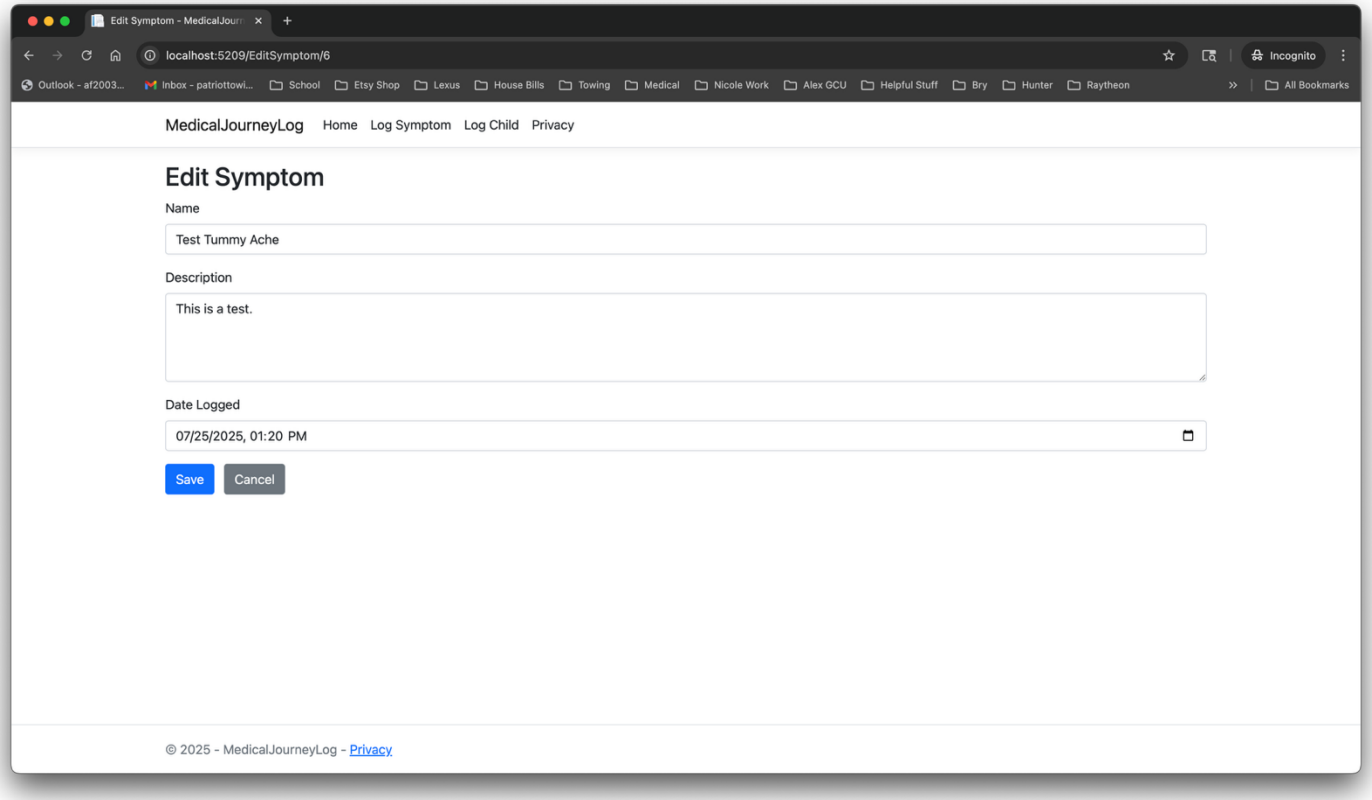

Figure 6 Screenshot showing the Edit Symptom form before changes are made, displaying the existing symptom details for the selected child.

Edit Symptom (Successful Update)

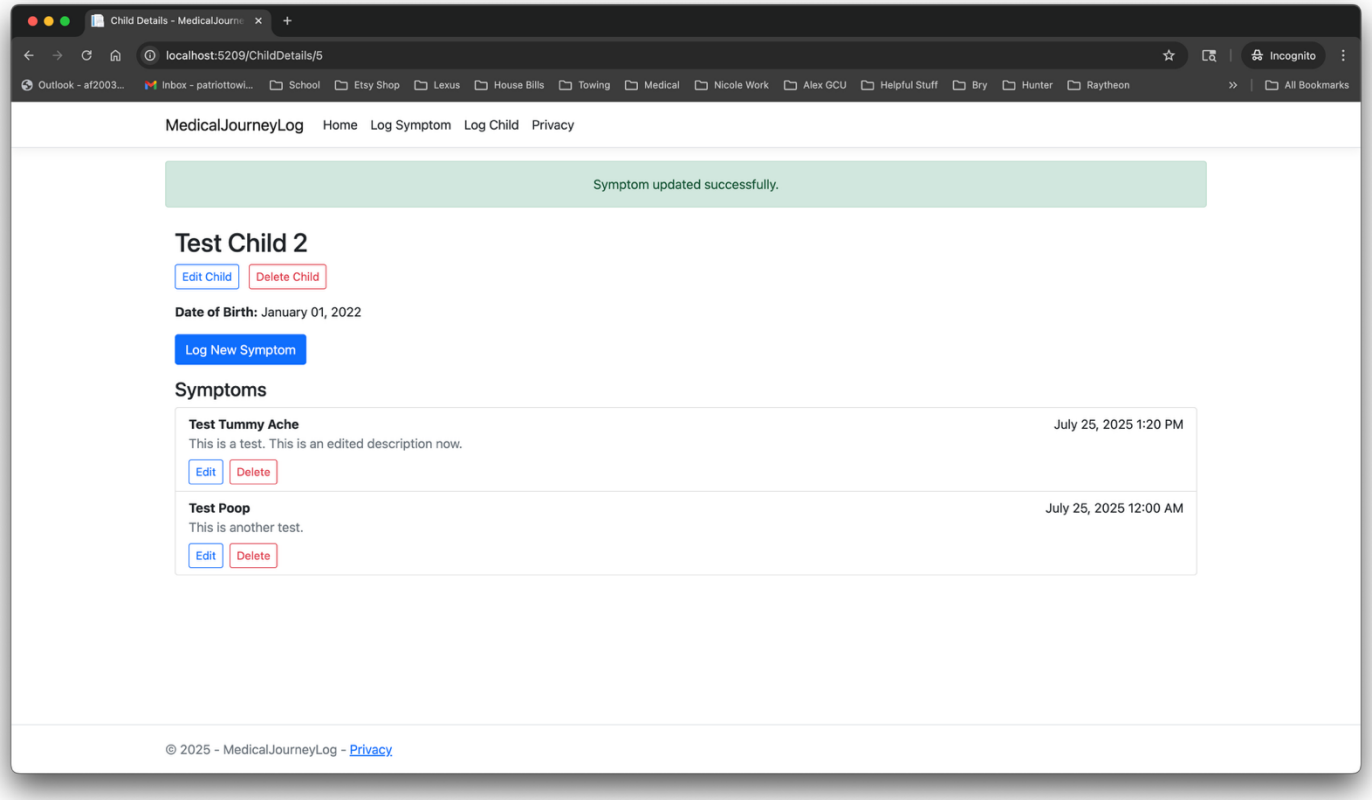

Figure 7 Screenshot showing a successful symptom update with the green success alert displayed at the top of the Child Details page. The updated symptom appears in the list.

Edit Symptom (Validation Error)

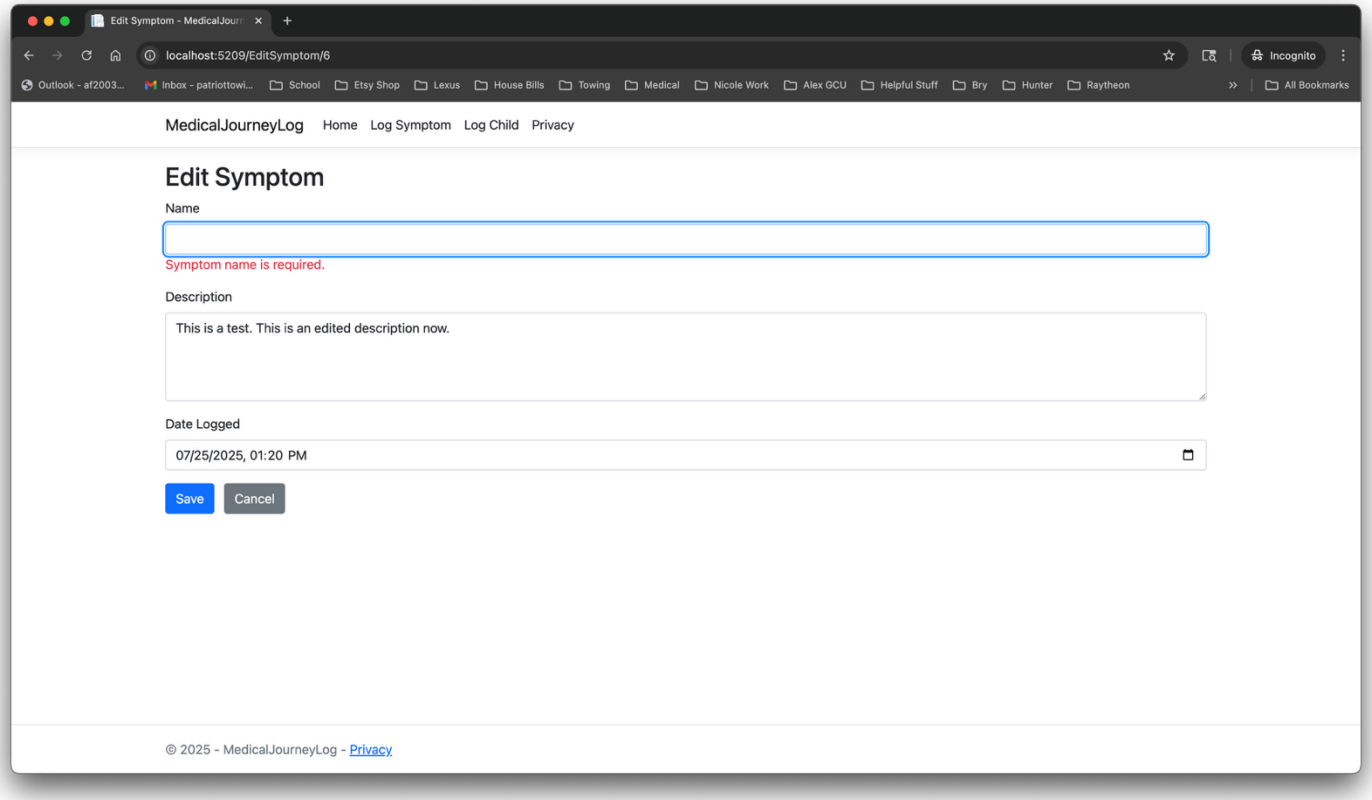

Figure 8 Screenshot showing the validation error message displayed when the required Symptom Name field is left blank. The page prevents saving until the input is corrected.

Test Case TC-008 – Delete Symptom

Field	Value
Objective	Verify caregiver can delete a symptom from a child's record.
Pre-Conditions / Data	Child with at least one Symptom in DB.
Steps	1. From **ChildDetails**, click **Delete** on a symptom 2. Confirm Delete.
Expected	Redirect to **/ChildDetails/{ChildId}**; success alert displayed; symptom removed from DB.
Evidence	Screenshots before and after deletion.
Status	Pass / Fail

Delete Symptom (Before Deletion)

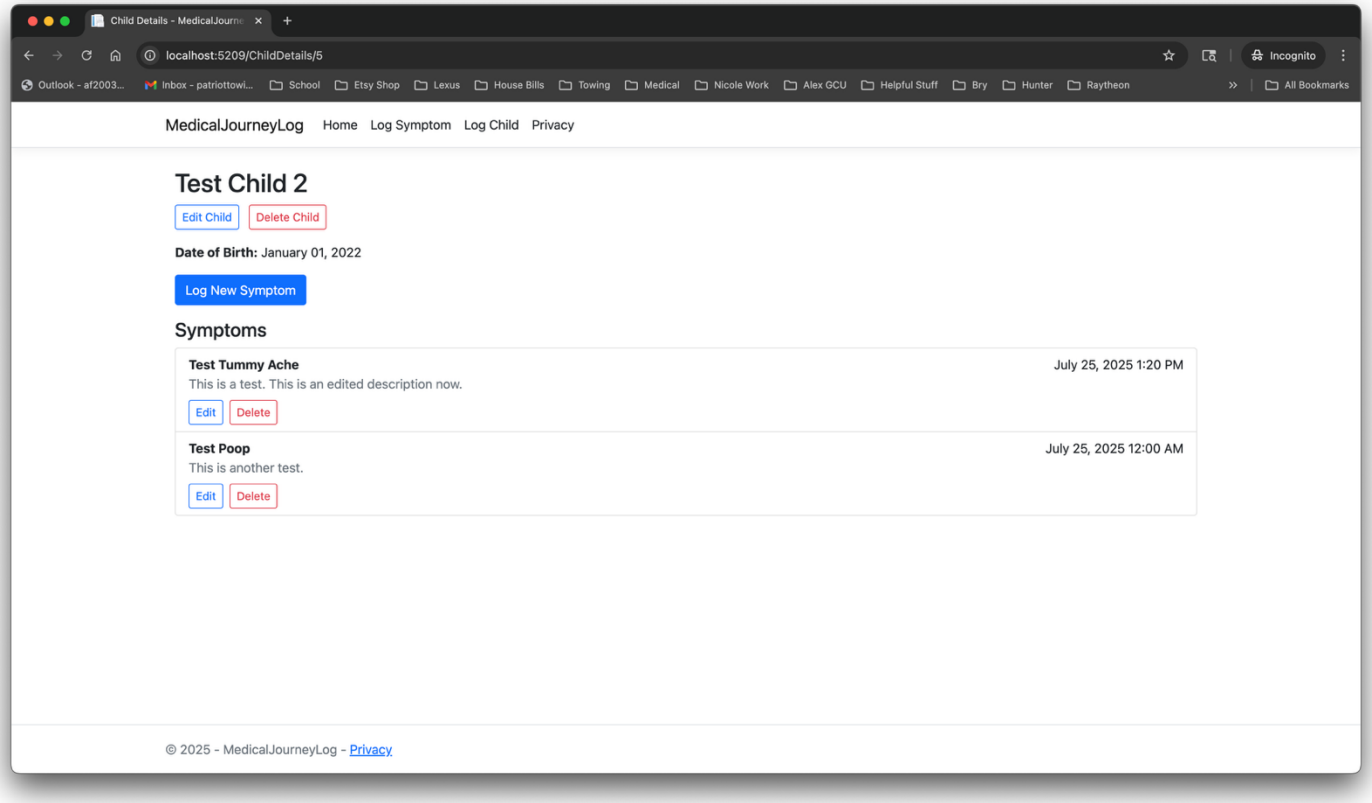

Figure 9 Screenshot showing the Child Details page before deleting a symptom. The list displays all existing symptoms with corresponding Edit and Delete controls.

Delete Symptom (Successful Deletion)

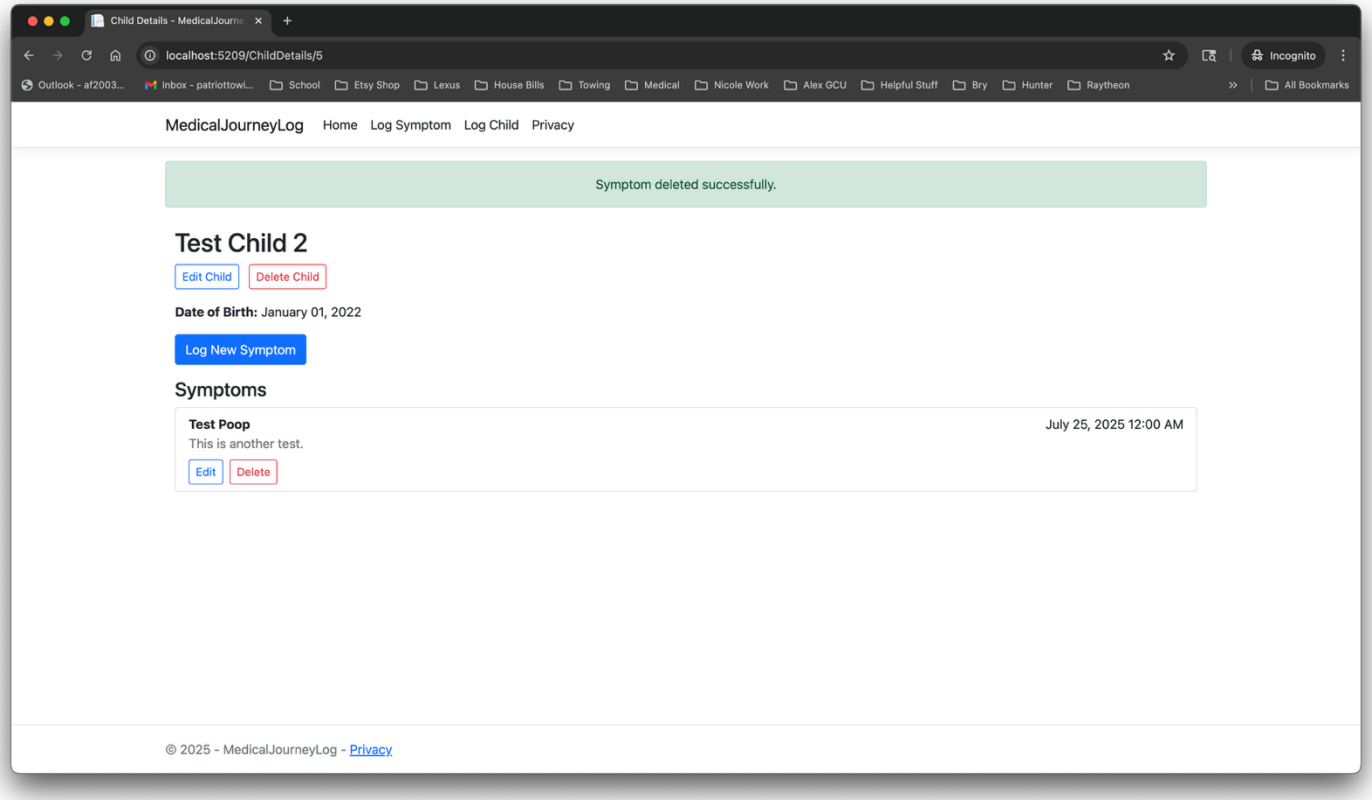

Figure 10 Screenshot showing confirmation of successful symptom deletion. The success alert appears at the top of the page, and the removed symptom no longer appears in the list.

4. Unit Test Plan

The Unit Test Plan defines how individual components (Razor Page handlers, model validation rules, and EF Core operations) are tested in isolation. Unit tests verify that methods and validation annotations behave as expected before being integrated into full workflows.

Unit Test Strategy

- **Scope:** PageModel `OnGetAsync` and `OnPostAsync` handlers for Edit/Delete (Child, Symptom), and model validation attributes in `Child.cs` and `Symptom.cs`.
- **Technique:** Manual execution with test inputs, plus review of server-side validation messages.
- **Tools:** Visual Studio debugger, browser-based form submissions, EF Core logging.
- **Responsibility:** Developer (Alex Frear).
- **Environment:** Localhost environment on MacBook Pro, .NET 7, MySQL (MAMP).

Unit Test Cases

Unit Under Test	Test ID	Description	Input	Expected Output
`Child` model validation	U-001	Verify Name is required	Name = ""	ModelState invalid, error "Name is required."
`Child` model validation	U-002	Verify Name max length 80	Name = 81-char string	ModelState invalid, error "Name cannot be longer than 80 characters."
`Symptom` model validation	U-003	Verify Name is required	Name = ""	ModelState invalid, error "Symptom name is required."
`Symptom` model validation	U-004	Verify Description max length 500	Description = 600 chars	ModelState invalid, error "Description cannot exceed 500 characters."
`EditChild.OnPostAsync`	U-005	Successful update	Valid Name + DOB	Redirects to ChildDetails with success TempData
`DeleteChild.OnPostAsync`	U-006	Block delete with symptoms	Child has ≥1 symptom	Redirects to ChildDetails with error TempData
`EditSymptom.OnPostAsync`	U-007	Successful update	Valid Name + Description	Redirects to ChildDetails with success TempData
`DeleteSymptom.OnPostAsync`	U-008	Successful delete	Valid SymptomId	Redirects to ChildDetails; symptom removed

Entry and Exit Criteria

- **Entry:** Component implemented and builds successfully.
- **Exit:** All unit test cases pass with expected outcomes; no critical issues remain.

5. Integration Test Plan

The Integration Test Plan validates that multiple components of the Medical Journey Log system work together correctly. This includes Razor Page handlers, EF Core data access, and MySQL persistence. The focus is on end-to-end flows across the UI, server, and database.

Integration Test Strategy

- **Scope:** Flows involving children and symptoms — creation, editing, deletion, and validation.
- **Technique:** Manual black-box testing of workflows through the web interface; verifying persisted changes in the database.
- **Tools:** Browser (Chrome), EF Core logs, MySQL Workbench.
- **Responsibility:** Developer (Alex Frear).
- **Environment:** Localhost environment on MacBook Pro, .NET 7, MySQL (MAMP).

Integration Test Cases

Test ID	Scenario	Steps	Expected Outcome
I-001	Add → Edit → Delete Child (no symptoms)	1. Log a child 2. Edit Name 3. Delete child	Success alerts shown each step; DB updated accordingly
I-002	Add Symptom → Edit → Delete	1. Log symptom for child 2. Edit symptom name 3. Delete symptom	ChildDetails updates correctly; DB updates and removes row
I-003	Delete Child with Symptoms (blocked)	1. Log child 2. Add symptom 3. Try Delete Child	Blocked with error alert; DB unchanged
I-004	Validation integration	1. Try saving child with blank Name 2. Try saving symptom with long description	Validation messages displayed; DB unchanged

Entry and Exit Criteria

- **Entry:** Unit testing complete, all pages accessible.
- **Exit:** All integration cases pass; data is consistent between UI and database; no broken workflows.

6. System Test Plan

The System Test Plan evaluates the Medical Journey Log application as a whole, ensuring that all integrated features meet the specified requirements. This testing phase validates end-to-end workflows, user experience, and system-level behaviors across the project.

System Test Strategy

- **Scope:** Full caregiver workflows — adding, editing, and deleting children and symptoms, with proper validation and error handling.
- **Technique:** Manual end-to-end testing of features in sequence, simulating real caregiver usage.
- **Tools:** Web browser (Chrome), MySQL Workbench for backend verification.
- **Responsibility:** Developer (Alex Frear).
- **Environment:** Localhost environment on MacBook Pro, .NET 7, MySQL (MAMP).

System Test Cases

Test ID	Scenario	Steps	Expected Outcome
S-001	Full lifecycle of child and symptom	1. Log a new child 2. Add symptom 3. Edit child 4. Edit symptom 5. Delete symptom 6. Delete child	All steps succeed; success/error alerts displayed; DB reflects changes
S-002	Validation enforcement	1. Attempt to log child with blank name 2. Attempt to log symptom with blank name 3. Attempt to enter >80 char name for child	Validation errors shown; no invalid DB entries
S-003	Delete guard workflow	1. Log child 2. Add symptom 3. Attempt delete child	System blocks deletion; error alert shown; DB unchanged

Entry and Exit Criteria

- **Entry:** Integration testing complete; application stable on localhost.
- **Exit:** All system test cases executed successfully; application demonstrates expected caregiver workflows without critical issues.

7. Test Case Specifications

The following specifications provide detailed definitions for the new test cases introduced in Iteration 3. Each specification includes inputs, expected outputs, and dependencies.

TC-005: Edit Child

- **Objective:** Verify caregiver can edit a child's details.
- **Preconditions:** Child record exists in database.
- **Inputs:** Valid and invalid values for `Name` and `DateOfBirth`.
- **Expected Results:**
  - Valid inputs → child updated, success alert displayed, DB updated.
  - Invalid inputs (blank name, name >80 chars) → validation errors shown, DB unchanged.
- **Dependencies:** `Child` model annotations, `EditChild` page.

TC-006: Delete Child (Guarded)

- **Objective:** Verify caregiver can delete a child only if no symptoms exist.
- **Preconditions:** Case A: Child with 0 symptoms. Case B: Child with ≥1 symptom.
- **Inputs:** Child ID.
- **Expected Results:**
  - Case A → child deleted, success alert, DB row removed.
  - Case B → deletion blocked, error alert, DB unchanged.
- **Dependencies:** `DeleteChild` page, EF Core guard check on symptoms.

TC-007: Edit Symptom

- **Objective:** Verify caregiver can edit an existing symptom.
- **Preconditions:** Symptom exists for a child in DB.
- **Inputs:** Valid and invalid values for `Name` and `Description`.
- **Expected Results:**
  - Valid inputs → symptom updated, success alert, DB updated.
  - Invalid inputs (blank name, description >500 chars) → validation errors, DB unchanged.
- **Dependencies:** `Symptom` model annotations, `EditSymptom` page.

TC-008: Delete Symptom

- **Objective:** Verify caregiver can delete an existing symptom.
- **Preconditions:** Symptom exists for a child in DB.
- **Inputs:** Symptom ID.
- **Expected Results:** Symptom deleted, success alert, removed from DB and no longer visible on Child Details.
- **Dependencies:** `DeleteSymptom` page, EF Core deletion.

8. Test Analysis Approval Form

The following approval form documents that the test analysis for Iteration 3 has been completed. The listed features have been implemented, tested, and reviewed according to the CST-452 Milestone 4 requirements.

Test Analysis Approval Form – Iteration 3 (Final Release)

Feature ID	Description	Test Case(s)	Status
F1	Edit Child	TC-005	Pass
F2	Delete Child (guarded)	TC-006	Pass
F3	Edit Symptom	TC-007	Pass
F4	Delete Symptom	TC-008	Pass
F5	Validation Enforcement	TC-005 to TC-008	Pass

All features F1–F5 passed regression testing. Application verified and ready for submission.

9. Source Code Listing – Final Iteration (Iteration 3)

No major code changes from Iteration 2. Source verified and maintained in GitHub repository.

Updated Model Files

Child.cs

```
using System.ComponentModel.DataAnnotations;

namespace MedicalJourneyLog.Models
{
    public class Child
    {
        public int Id { get; set; }

        [Required(ErrorMessage = "Name is required.")]
        [StringLength(80, ErrorMessage = "Name cannot be longer than 80
characterspublic
        public string Name { get; set; } = string.Empty;

        [DataType(DataType.Date)]
        [Display(Name = "Date of Birth")]
        public DateTime? DateOfBirth { get; set; }

        public ICollection<Note>? Notes { get; set; }
        public ICollection<Symptom>? Symptoms { get; set; }
        public ICollection<Milestone>? Milestones { get; set; }
        public ICollection<Appointment>? Appointments { get; set; }
        public ICollection<CaregiverChild>? CaregiverChildren { get; set; }
    }
}
```

Symptom.cs

```
using System.ComponentModel.DataAnnotations;

namespace MedicalJourneyLog.Models
{
    public class Symptom
    {
        public int Id { get; set; }

        [Required(ErrorMessage = "Symptom name is required.")]
        [StringLength(80, ErrorMessage = "Symptom name cannot be longer than 80
characterspublic
        public string Name { get; set; } = string.Empty;

        [StringLength(500, ErrorMessage = "Description cannot exceed 500 characters.")]
        public string? Description { get; set; }

        [DataType(DataType.DateTime)]
        [Display(Name = "Date Logged")]
        public DateTime DateLogged { get; set; } = DateTime.Now;

        [Required]
        public int ChildId { get; set; }
        public Child? Child { get; set; }
    }
}
```

New Razor Pages

EditChild.cshtml.cs

```
using MedicalJourneyLog.Data;
using MedicalJourneyLog.Models;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using Microsoft.EntityFrameworkCore;

namespace MedicalJourneyLog.Pages
{
    public class EditChildModel : PageModel
    {
        private readonly ApplicationDbContext _context;
        public EditChildModel(ApplicationDbContext context) => _context = context;

        [BindProperty]
        public Child Child { get; set; } = new();

        public async Task<IActionResult> OnGetAsync(int id)
        {
            var child = await _context.Children.FirstOrDefaultAsync(c => c.Id == id);
            if (child is null) return NotFound();
            Child = child;
            return Page();
        }

        public async Task<IActionResult> OnPostAsync(int id)
        {
            if (!ModelState.IsValid) return Page();

            var childToUpdate = await _context.Children.FirstOrDefaultAsync(c => c.Id ==
id);        if (childToUpdate is null) return NotFound();

            childToUpdate.Name = Child.Name;
            childToUpdate.DateOfBirth = Child.DateOfBirth;

            await _context.SaveChangesAsync();

            TempData["SuccessMessage"] = "Child updated successfully.";
            return RedirectToPage("/ChildDetails", new { id });
        }
    }
}
```

DeleteChild.cshtml.cs

```
import React from 'react';

const Card = (pusing MedicalJourneyLog.Data;
using MedicalJourneyLog.Models;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using Microsoft.EntityFrameworkCore;

namespace MedicalJourneyLog.Pages
{
    public class DeleteChildModel : PageModel
    {
        private readonly ApplicationDbContext _context;
        public DeleteChildModel(ApplicationDbContext context) => _context = context;

        [BindProperty]
        public Child Child { get; set; } = new();

        public string? BlockReason { get; set; }

        public async Task<IActionResult> OnGetAsync(int id)
        {
            var child = await _context.Children
                .Include(c => c.Symptoms)
                .FirstOrDefaultAsync(c => c.Id == id);

            if (child is null) return NotFound();

            Child = child;

            if (child.Symptoms != null && child.Symptoms.Any())
            {
                BlockReason = "This child has one or more symptoms. Please delete the symptoms first
before deleting the child.";
            }

            return Page();
        }

        public async Task<IActionResult> OnPostAsync(int id)
        {
            var child = await _context.Children
                .Include(c => c.Symptoms)
                .FirstOrDefaultAsync(c => c.Id == id);

            if (child is null) return NotFound();

            if (child.Symptoms != null && child.Symptoms.Any())
            {
                TempData["ErrorMessage"] = "Delete blocked: this child has symptoms. Delete those
first.";
                return RedirectToPage("/ChildDetails", new { id });
            }

            _context.Children.Remove(child);
            await _context.SaveChangesAsync();

            TempData["SuccessMessage"] = "Child deleted successfully.";
            return RedirectToPage("/Index");
        }
    }
}
rops) => {
  return (
    <div className="card mb-4" style={{ width: '18rem' }}>
      <img src={props.imgURL} className="card-img-top" alt={props.albumTitle} />
      <div className="card-body">
        <h5 className="card-title">{props.albumTitle}</h5>
        <p className="card-text">{props.albumDescription}</p>
        <button className="btn btn-primary">{props.buttonText}</button>
      </div>
    </div>
  );
};

export default Card;
```

EditSymptom.cshtml.cs

```
using MedicalJourneyLog.Data;
using MedicalJourneyLog.Models;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using Microsoft.EntityFrameworkCore;

namespace MedicalJourneyLog.Pages
{
    public class EditSymptomModel : PageModel
    {
        private readonly ApplicationDbContext _context;
        public EditSymptomModel(ApplicationDbContext context) => _context = context;

        [BindProperty]
        public Symptom Symptom { get; set; } = new();

        public async Task<IActionResult> OnGetAsync(int id)
        {
            var symptom = await _context.Symptoms.FirstOrDefaultAsync(s => s.Id == id);
            if (symptom is null) return NotFound();
            Symptom = symptom;
            return Page();
        }

        public async Task<IActionResult> OnPostAsync(int id)
        {
            if (!ModelState.IsValid) return Page();

            var symptomToUpdate = await _context.Symptoms.FirstOrDefaultAsync(s => s.Id ==
id);        if (symptomToUpdate is null) return NotFound();

            symptomToUpdate.Name = Symptom.Name;
            symptomToUpdate.Description = Symptom.Description;
            symptomToUpdate.DateLogged = Symptom.DateLogged;

            await _context.SaveChangesAsync();

            TempData["SuccessMessage"] = "Symptom updated successfully.";
            return RedirectToPage("/ChildDetails", new { id = symptomToUpdate.ChildId });
        }
    }
}
```

DeleteSymptom.cshtml.cs

```
using MedicalJourneyLog.Data;
using MedicalJourneyLog.Models;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using Microsoft.EntityFrameworkCore;

namespace MedicalJourneyLog.Pages
{
    public class DeleteSymptomModel : PageModel
    {
        private readonly ApplicationDbContext _context;
        public DeleteSymptomModel(ApplicationDbContext context) => _context = context;

        [BindProperty]
        public Symptom Symptom { get; set; } = new();

        public async Task<IActionResult> OnGetAsync(int id)
        {
            var symptom = await _context.Symptoms.FirstOrDefaultAsync(s => s.Id ==
id);        if (symptom is null) return NotFound();
            Symptom = symptom;
            return Page();
        }

        public async Task<IActionResult> OnPostAsync(int id)
        {
            var symptom = await _context.Symptoms.FirstOrDefaultAsync(s => s.Id ==
id);        if (symptom is null) return NotFound();

            var childId = symptom.ChildId;

            _context.Symptoms.Remove(symptom);
            await _context.SaveChangesAsync();

            TempData["SuccessMessage"] = "Symptom deleted successfully.";
            return RedirectToPage("/ChildDetails", new { id = childId });
        }
    }
}
```

Updated Razor Page

ChildDetails.cshtml

```
@page "{id:int}"
@model MedicalJourneyLog.Pages.ChildDetailsModel
@{
    ViewData["Title"] = "Child Details";
}

<div class="container mt-4">
    @if (TempData["SuccessMessage"] is string ok && !string.IsNullOrWhiteSpace(ok))
    {
        <div class="alert alert-success">@ok</div>
    }
    @if (TempData["ErrorMessage"] is string err && !string.IsNullOrWhiteSpace(err))
    {
        <div class="alert alert-danger">@err</div>
    }

    <h2>@Model.Child.Name</h2>

    <div class="mb-3">
        <a asp-page="/EditChild" asp-route-id="@Model.Child.Id" class="btn btn-outline-primary btn-
sm">Edit Child</a>
        <a asp-page="/DeleteChild" asp-route-id="@Model.Child.Id" class="btn btn-outline-danger btn-sm
ms-2">Delete Child</a>
    </div>

    <p>
        <strong>Date of Birth:</strong>
        @Model.Child.DateOfBirth?.ToString("MMMM dd, yyyy")
    </p>

    <a class="btn btn-primary mb-3"
       asp-page="/LogSymptom"
       asp-route-childId="@Model.Child.Id">
        Log New Symptom
    </a>

    <h4>Symptoms</h4>

    @if (Model.Child.Symptoms != null && Model.Child.Symptoms.Any())
    {
        <ul class="list-group">
            @foreach (var symptom in Model.Child.Symptoms)
            {
                <li class="list-group-item">
                    <div class="d-flex justify-content-between align-items-start">
                        <div>
                            <strong>@symptom.Name</strong>
                            @if (!string.IsNullOrWhiteSpace(symptom.Description))
                            {
                                <div class="text-muted">@symptom.Description</div>
                            }
                        </div>
                        <div class="text-end">
                            <time datetime="@symptom.DateLogged.ToString("o")">
                                @symptom.DateLogged.ToString("MMMM dd, yyyy h:mm tt")
                            </time>
                        </div>
                    </div>

                    <div class="mt-2 d-flex gap-2">
                        <a asp-page="/EditSymptom"
                           asp-route-id="@symptom.Id"
                           class="btn btn-outline-primary btn-sm">Edit</a>

                        <form method="post"
                              asp-page="/DeleteSymptom"
                              asp-route-id="@symptom.Id"
                              onsubmit="return confirm('Delete this symptom?');"
                              class="d-inline">
                            <button type="submit" class="btn btn-outline-danger btn-sm">Delete</button>
                        </form>
                    </div>
                </li>
            }
        </ul>
    }
    else
    {
        <p>No symptoms recorded yet.</p>
    }
</div>
```

10. Reflection / Conclusion

The Medical Journey Log project achieved full implementation of all functional requirements. Final testing confirmed that all modules operate as intended, providing caregivers with a reliable way to record children's symptoms and updates. Future enhancements will include caregiver authentication, shared access, and milestone tracking.